# CS 223

Week 6 Lecture 1



# Grade Breakdown

Final Exam: 30%

Quizzes: 35%

Homework: 15%

Labs: 15%

Attendance / Participation: 5%

# Announcements

Lab today → will instead be assigned as homework (due after fall break)

Today:

- Binary
- Overflow
- Binary addition (and overflow)

Enjoy your fall break! :)

# Aside: Coding with powers of 2

Multiplying and dividing by 2 corresponds to shifting bits left and right

Bit shift operators:

- << shift bits to the left
- >> shift bits to the right

Ex: pow(2,3) can be computed as 1 << 3

Ex: x = x * 2 corresponds to x = x << 1

# Exercise: Write a program that outputs binary from decimal

Algorithm: Repeatedly divide by 2 until the input is 0

```
$ ./binary
Enter a positive decimal number: 422
422, 0
211, 1
105, 1
52, 0
26, 0
13, 1
6, 0
3, 1
1, 1
```

# Sizes of data types

Need to decide how many bits we should use to store different types

What factors should we consider?

# Unsigned integers

Numbers ranging from 0 to a positive max value

What is the largest number that can be stored in 4 bits?

What is the largest number that can be stored in 4 bytes?

# Overflow

When we try to store a value too large to fit into a data type, we get **overflow**

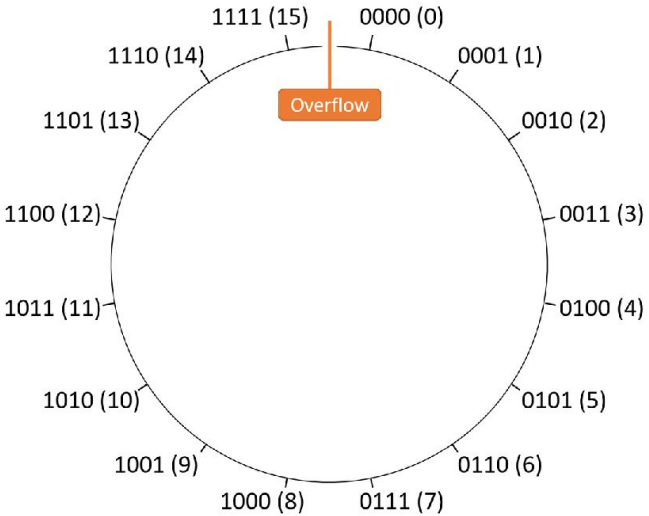



*Figure 3. An arrangement of four-bit unsigned values into a modular space. All arithmetic is modular with respect to $2^4$ (16).*

# What is the output of this program?

```
#include <stdio.h>

int main() {
   unsigned int a = 0;
   for (a = 5; a >= 0; a--) {
      printf("Message!\n");
   }
   return 0;
}
```

# Signed integers

Modern systems use a method called **two's complement**

Highest order bit encodes the sign (0 -> positive; 1 -> negative)

Advantage: allows pos/neg numbers to be treated the same in hardware

# Two’s complement

Suppose we have N bits to represent a signed integer. The formula is

$$-(d_{N-1} \times 2^{N-1}) + (d_{N-2} \times 2^{N-2}) + \ldots + (d_2 \times 2^2) + (d_1 \times 2^1) + (d_0 \times 2^0)$$

^ note the leading negative sign for just the first term!

Example: What is 1001 interpreted as a *signed* integer? As an *unsigned* integer?

# Exercise

Convert the following *signed* integer to decimal: 0b 0110

Convert the following *signed* integer to decimal: 0b 1111

# Two’s complement: 4 bits

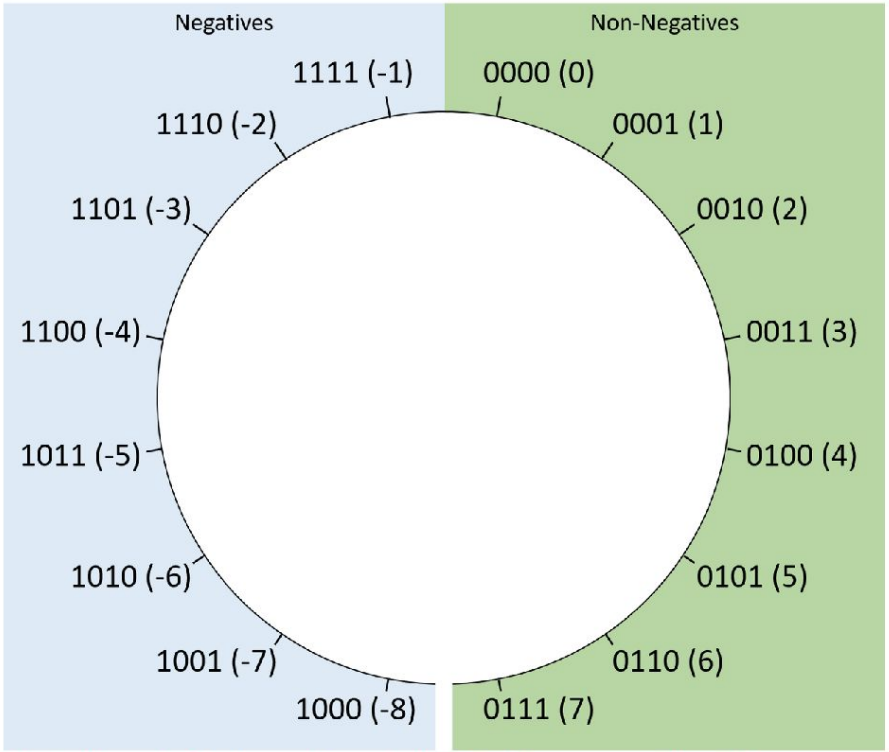



*Figure 36. A logical layout of two’s complement values for bit sequences of length four.*

# Question

If we use N bits, what is the range of unsigned integers we can represent?

If we use N bits, what is the range of signed integers we can represent?

# Negation

To negate a two's complement signed integer

flip the bits

add one

Example: Compute -5 from 5 in binary

# Why the name two's complement?

$$X + (-X) = 2^N$$

For N bit numbers

# Binary addition

Works like decimal addition: we carry over values when we reach our max digit.

Example: Add 2 + 8 as 4 bit unsigned binary numbers

# Binary addition: overflow

What happens when we run out of bits? **truncation**

Example: Add 12 + 7 as unsigned 4 bit numbers

# Visualizing Overflow

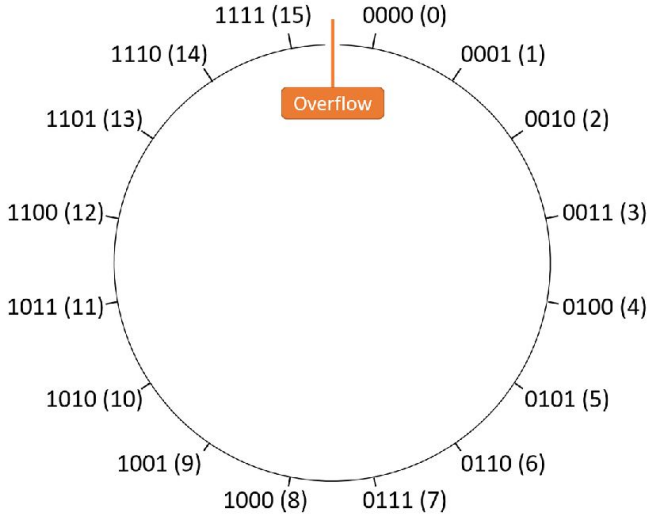



*Figure 3. An arrangement of four-bit unsigned values into a modular space. All arithmetic is modular with respect to $2^4$ (16).*

# Binary addition

Possible outcomes when considering two binary digits and 1 carry digit

| Inputs | | | | Outputs | |
|---|---|---|---|---|---|
| Digit A | Digit B | Carry in | | Sum | Carry out |
| 0 | 0 | 0 | | | |
| 0 | 0 | 1 | | | |
| 0 | 1 | 0 | | | |
| 0 | 1 | 1 | | | |
| 1 | 0 | 0 | | | |
| 1 | 0 | 1 | | | |
| 1 | 1 | 0 | | | |
| 1 | 1 | 1 | | | |

# Example: Add 8 + 3

Add 8 + 3 as unsigned 4-bit numbers

# Example: Add 3 + 3

Add 3 + 3 as unsigned 4-bit numbers

# Binary subtraction

Idea: X – Y is the same as X + (-Y)

Therefore, negate the second operand and then add

Example: Compute 7-3 as 4 bit signed integers

# Signed extension

What happens when you perform an arithmetic operation on numbers with different sizes? **signed extension**

For *unsigned* values, prepend 0

For *signed* values, prepend the leftmost bit

# Example: Signed extension

Add 5 and -5 as 8 bit numbers

# Signed extension: example

| | |
|---|---|
| 0b 1011 | |
| 0b 1 1011 | |
| 0b 11 1011 | |

# Floating point

Goals of representation

- Need to encode the sign
- Need to encode both very large and very small numbers

Idea: We let the placement of the decimal point change

# Floating point

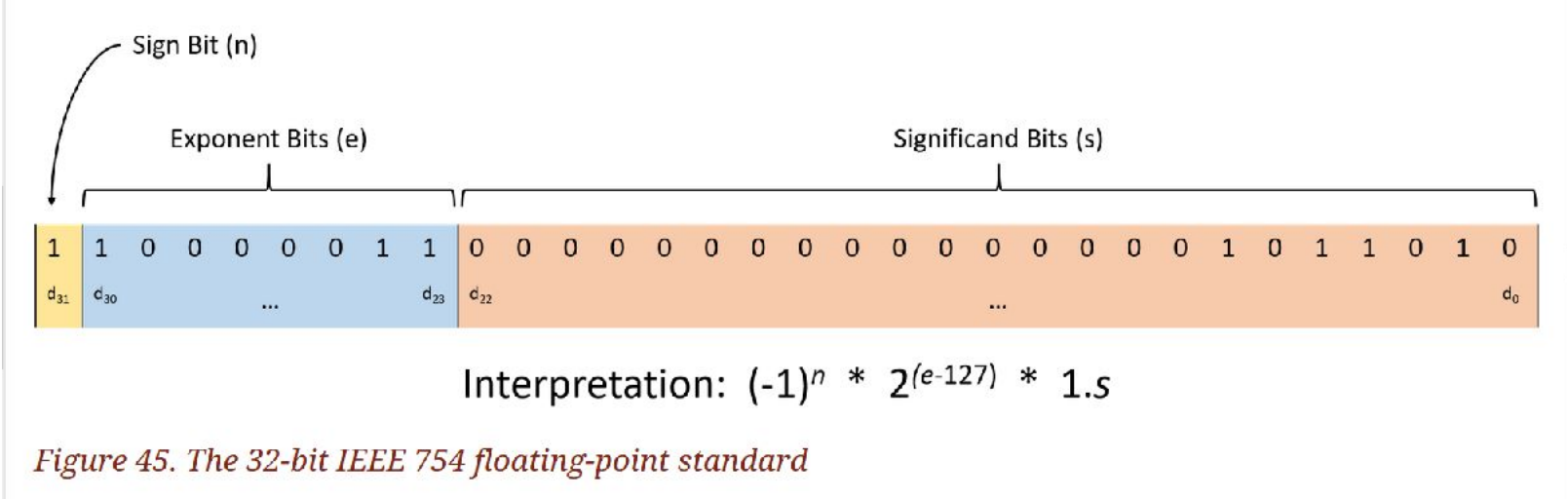



*Figure 45. The 32-bit IEEE 754 floating-point standard*

NOTE: e-127 called the **bias,** range is [-126,127]

# Floating point example

What is the value of this binary floating point number in decimal?

1100 0001 1011 0100 0000 0000 0000 0000